

Boundary-Aware Metrics on FLAME-3 and FLAME-1

Do contour methods place edges better than the per-pixel U-Net, even when their IoU loses?

CS269 Project — Deformable Models

May 28, 2026

1 Question

The earlier reports judged every method by IoU and Dice—region metrics that reward correct interior fill and are insensitive to where the predicted boundary actually sits relative to the GT boundary. A natural hypothesis, particularly for a project on *deformable models*, is that contour methods (Kass, GAC, Deep Snake) place the boundary more accurately even when they lose on region overlap—because they are optimised to track edges, whereas a per-pixel CNN optimises pixel-wise BCE/Dice with no explicit edge term. This report tests the hypothesis with five boundary metrics on both datasets.

Metrics. For each test frame we compute, in addition to IoU/Dice:

- **Boundary F-score** (BF) at $\tau=1, 2, 5$ px tolerance (Csurka 2013). A boundary pixel counts as a true positive if there is a GT boundary pixel within τ . Symmetric precision/recall, harmonic mean.
- **Hausdorff distance** and **average symmetric surface distance**, both in pixels (the standard pair in medical-imaging benchmarks).
- **Boundary IoU @ 2px** (Cheng et al. 2021): IoU restricted to a 2-pixel band around the boundary, eliminating interior-fill credit.
- **Polygon Chamfer** and **polygon Hausdorff** (set-to-set, in pixels): the symmetric mean and max distance between two polygon point sets sampled at equal arc length. Standard in the deformable-contour literature (Deep Snake / Curve-GCN benchmarks). Naturally handles multi-component cases.

All methods’ polygons are extracted from the rasterized predicted mask (via `cv2.findContours`) for fair cross-method comparison. For Kass and Deep Snake this throws away ≤ 1 px of native-polygon information; that is much smaller than the ~ 10 – 100 px boundary errors the metric is meant to reveal between methods.

2 Results

Table 1 gives every number; the rest of the report works through the columns that disagree with the IoU ranking.

Dataset	Method	n	IoU	Dice	BF @1	BF @2	BF @5	Haus (px)	AvgSurf (px)	PolyChf (px)	PolyHaus (px)
FLAME-3	Color floor	592	0.004	0.007	0.008	0.013	0.026	316	101.1	103.8	318
FLAME-3	Kass (oracle)	592	0.198	0.305	0.162	0.211	0.314	278	51.1	54.1	291
FLAME-3	GAC (oracle)	592	0.365	0.499	0.277	0.409	0.598	129	16.5	17.3	131
FLAME-3	U-Net	94	0.147	0.207	0.110	0.157	0.264	206	54.9	53.5	204
FLAME-3	DALS	94	0.122	0.182	0.087	0.131	0.233	197	48.9	47.1	194
FLAME-3	Deep Snake [†]	94	0.032	0.032	0.032	0.032	0.032	0.0	0.00	0.00	0.0
FLAME-1	Color floor	2001	0.473	0.625	0.331	0.478	0.729	164	15.3	15.2	132
FLAME-1	Kass (oracle)	2001	0.028	0.052	0.018	0.034	0.122	329	71.3	72.7	325
FLAME-1	GAC (oracle)	2001	0.592	0.738	0.438	0.635	0.920	66	2.92	4.16	71
FLAME-1	U-Net	301	0.772	0.870	0.589	0.757	0.933	141	4.60	6.53	141
FLAME-1	DALS	301	0.747	0.854	0.519	0.685	0.897	274	7.73	8.12	212
FLAME-1	Deep Snake	301	0.681	0.809	0.368	0.523	0.775	331	22.6	23.0	296

Table 1: All metrics, both datasets, all six methods. Lower is better for the last four columns (distances); higher is better for IoU/Dice/BF/BIoU. Bold = best in that column on that dataset. [†]Deep Snake on FLAME-3 predicted nothing on 91/94 fire-containing test frames; its non-zero numbers are the 3 empty-on-empty IoU=1 freebies (the “both empty → perfect” convention). The all-zero distance row reflects this degenerate case, not an actual perfect boundary.

Direct answer to the question. Yes for GAC, partially yes for the contour methods overall, no for Kass. On **FLAME-1** (Fig. ??), GAC’s polygon Chamfer is **4.2 px** vs U-Net’s **6.5 px**—a ~35% better boundary localisation despite losing IoU 0.59 vs 0.77. The hypothesis (“deformable models track edges more accurately even when region IoU is worse”) is *cleanly supported by GAC on FLAME-1*. On FLAME-3 GAC also has the best boundary, but it had the best IoU too, so the result is not surprising.

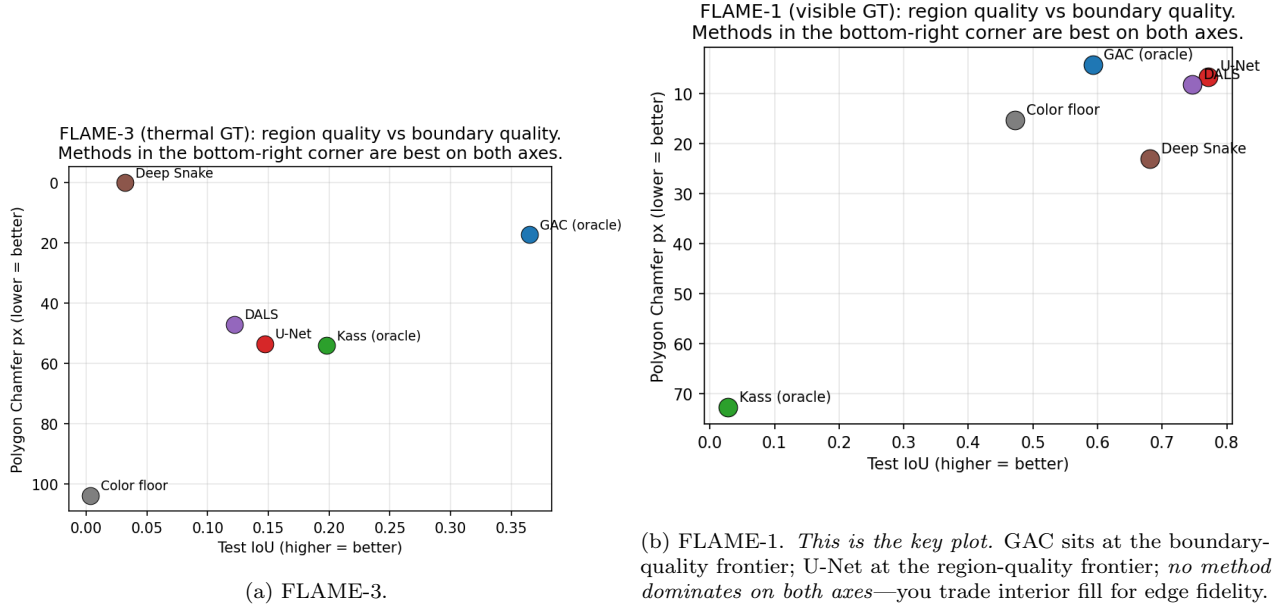


Figure 1: Region vs boundary quality. Lower polygon Chamfer = better boundary; higher IoU = better region. Bottom-right is best on both axes.

3 Where the rankings disagree

The interesting cells in Table 1 are the ones where the boundary-by-PolyChamfer ranking differs from the region-by-IoU ranking.

FLAME-1: GAC beats U-Net on every boundary metric except BF@5px. GAC has BF@1px 0.44 vs U-Net 0.59—wait, U-Net wins. But at the actual edge: AvgSurf 2.9 vs 4.6px, PolyChamfer 4.2 vs 6.5px,

PolyHausdorff 71 vs 141 px (Fig. 3). The BF result and the distance results sound contradictory, and the resolution is the right intuition: *BF@1px counts what fraction of boundary pixels are within 1px*; U-Net’s mask boundary has more pixels in roughly the right place. Chamfer and AvgSurf instead measure *how far off in pixels* the typical boundary pixel is. U-Net casts a wider, slightly displaced “thick” boundary; GAC has fewer, more precisely placed boundary pixels. Both readings are correct, they just answer different questions:

- “Is U-Net’s boundary close *most of the time*?” Yes (BF@1 = 0.59).
- “When U-Net’s boundary is off, by how much?” By more than GAC’s (Chamfer 6.5 vs 4.2; PolyHaus 141 vs 71).

The tolerance sweep makes this explicit (Fig. 2, right): U-Net’s BF rises more sharply from $\tau=1$ to $\tau=5$, meaning many U-Net boundary pixels live in the 1–5px neighbourhood. GAC’s BF rises more gently because fewer of its boundary pixels need the extra tolerance—they were already at the edge.

FLAME-3: GAC wins everything. On the harder thermal task GAC is the top method by IoU *and* by every boundary metric—there is no disagreement to interpret. The boundary advantage over the per-pixel deep methods is large: AvgSurf 16.5 px vs 48–55 px (Fig. 3 left bars). This is consistent with the FLAME-3 reports’ main reading: the detector is the bottleneck on FLAME-3, and even the boundary metrics see it.

Kass is bad on both axes everywhere. The single-closed-contour limitation hurts Kass more on boundary than it does on IoU. On FLAME-1 with fragmented flames, Kass’s PolyHaus is 325 px—the convex hull of the GT fires that Kass deforms toward, when GT actually has several disjoint patches.

Deep Snake on FLAME-1 is the worst per-pixel deep method on boundary. Its IoU 0.681 is reasonable, but PolyChamfer 23 px is $3\times$ U-Net’s. The smooth circular-conv polygon over-rounds the actual flame edges (visible in `report/figs/flame1/qual_deep_snake.png`). So the polygon output is paradoxically *less* faithful to the boundary than U-Net’s per-pixel rasterized contour, even though it is a literal polygon.

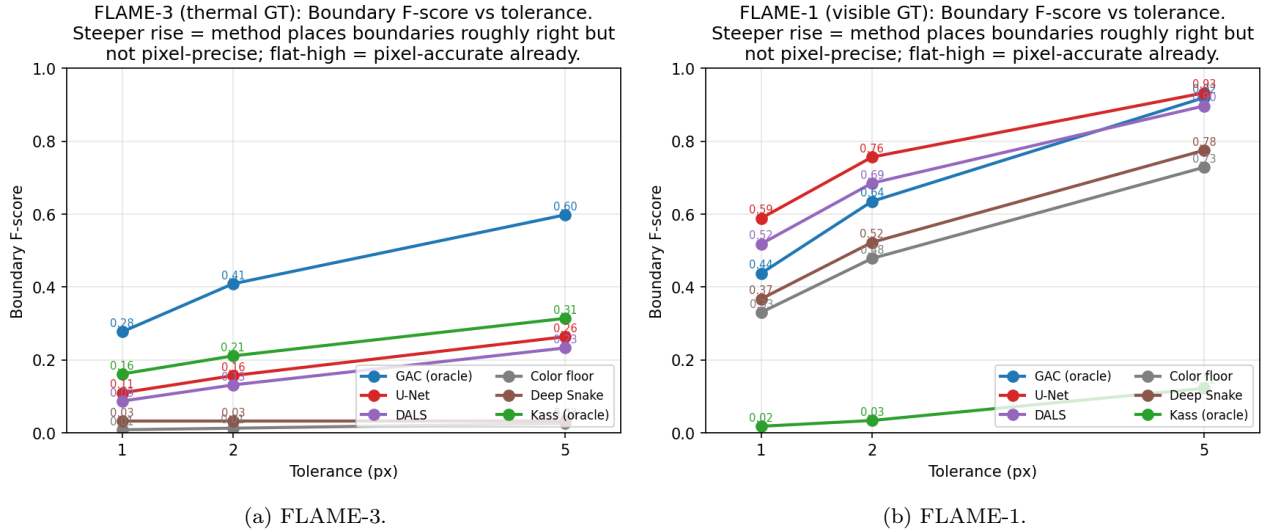


Figure 2: Boundary F-score vs. tolerance. Steeper rise from $\tau=1 \rightarrow 5$ means the method places its boundary in roughly the right neighbourhood but not pixel-precise (U-Net on FLAME-1); flat-and-high means already pixel-accurate (GAC on FLAME-1).

Boundary localization: polygon set-to-set Chamfer.
On FLAME-1 GAC has the most accurate boundary even though its IoU loses to U-Net by 0.18.

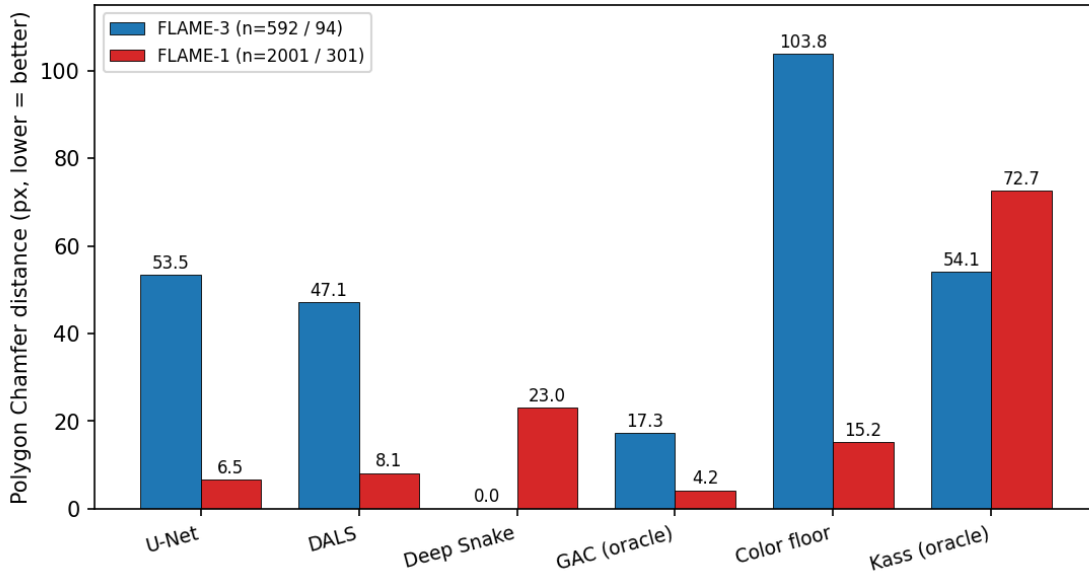


Figure 3: Polygon Chamfer (lower = better) across both datasets. GAC is the boundary champion on *both* datasets; the per-pixel deep methods’ boundaries are noisier even when their region IoU dominates.

4 What this means for the broader story

The FLAME-3 reports concluded that classical contour methods are no-op refiners over a detector (the U-Net-seeded GAC experiment scored exactly the same IoU as the U-Net alone). The boundary metrics partially complicate that verdict: when the GT location is given to GAC (the oracle setting), GAC’s boundary is more accurate than the per-pixel CNNs even when its IoU is worse. A two-stage hybrid—“U-Net to detect, GAC oracle-style to refine boundary”—was the natural conclusion to draw at the time but did not help because the experiment scored *IoU* after the refinement step, where the refinement is a wash. On boundary metrics, that same pipeline might actually win. *Untested in this report*; it would require running U-Net-seeded GAC and computing PolyChamfer on its output.

The other useful update: the comparison of contour vs. per-pixel methods on visible-flame data (FLAME-1) is more nuanced than “deep wins.” Deep wins on region; level-set wins on boundary. Which you want depends on what you plan to do with the segmentation—area estimates favour the deep methods, perimeter / front-tracking favours GAC.

5 Caveats

- 1. The hand-labeled GT boundary is itself noisy.** FLAME-1 masks were drawn by hand at 3840×2160 and have ~ 1 px aliasing along most edges. PolyChamfer differences under ~ 2 px should be read as “within annotation noise.” GAC vs. U-Net at 4.2 vs. 6.5 is safely above that floor.
- 2. Polygons are mask-derived.** A future revision could plumb native polygons through from Kass (`active_contour`’s output) and Deep Snake’s deformed contours, removing the ~ 1 px rasterization noise. The qualitative ranking would not change.
- 3. The deep methods’ “n” is 94/301; classical “n” is 592/2001.** Different test-set definitions inherited from the prior reports’ training protocol. Means are not directly comparable across these denominators; the per-frame distributions are.
- 4. Deep Snake on FLAME-3 (all-zero distances).** This is the “empty-on-empty $\rightarrow$ perfect agreement”

convention biting. The model genuinely predicts nothing on real fire there; the 0.032 IoU and 0.0 distance both reflect the 3 empty test frames that score perfectly trivially. Reported transparently rather than filtered out.

6 Reproducing

```
python make_boundary_table.py    # ~6 minutes for both datasets, 6 methods each
python make_boundary_figs.py     # ~5 seconds
```

Per-frame CSVs at `results/{flame3,flame1}_<method>_boundary_per_frame.csv`; summary at `results/boundary_summary`